

## Quiz 3 Problem Set

Quiz 3 will be based these parts of the textbook:

- Chapter 8: 8.1, 8.2, 8.3
- Chapter 9: 9.1, 9.2

The questions on the quiz may refer to definitions, theorems, and examples from the textbook. The questions could be of any type, and will be variations of questions below, or questions that are similar.

**Important:** *Treat these problem-sets as non-AI activities!* Turn off all AI support and try to figure them out yourself **by hand**. Having AI or another student do this for you will not help you learn. You must do the learning yourself!

### Question 1

- a) Write a **priority queue** abstract data type (ADT) using the operations below. For each operation, give the return-type and the type of the parameters (if any). Assume the items in the priority queue are all of type  $T$ , where  $T$  works with priority queue operations.

This is an ADT, so there are no implementation details: the operations have *no* implementation, and there are *no* variables.

Operations: *insert*, *remove\_min*, *min* (get copy of min), *size*, *is\_empty*

- b) Assuming the priority queue is implemented with a **linked list**, give an  $O$ -notation expression for the worst-case performance of the operations from part (a) in terms of  $n$  (the number of items in the priority queue).
- c) Assuming the priority queue is implemented with a **heap**, give an  $O$ -notation expression for the worst-case performance of the operations from part (a) in terms of  $n$  (the number of items in the priority queue).

### Question 2

- a) State the **heap order property** for a min heap.
- b) Suppose  $H$  is a min heap with many nodes. Its left subtree is called  $L$ , and its right subtree is called  $R$ . If you swap  $L$  and  $R$ , is  $H$  still a min heap?

### Question 3

Prove proposition 8.5 (the height of a heap) on p.399 from the textbook.

## Question 4

Using an array (not a vector<int>), implement a priority queue class based on a **min heap** of ints.

## Question 5

- In a heap implemented with an array, what is the index of the **parent** of the element with index  $i$ ? State clearly if the first element of the heap is at index 0 or index 1.
- In a heap implemented with an array, what are the indexes of the **children** of the element with index  $i$ ? State clearly if the first element of the heap is at index 0 or index 1.

## Question 6

Suppose you start with an empty **min heap** and insert these elements (in the order given) into the heap: 17, 12, 5, 8, 10, 3, 7, 4

- Sketch the **min heap** that results after doing all the insertions.
- Sketch the heap after calling *remove\_min* one time.

Be sure to practice doing this by hand on paper! For more practice, re-do this question using these values:

- 1, 2, 3, 4, 5, 6, 7, 8, 9
- 9, 8, 7, 6, 5, 4, 3, 2, 1

## Question 7

A polynomial hash code for a string can be calculated using the formula:

$x_0a^{k-1} + x_1a^{k-2} + \dots + x_{k-2}a + x_{k-1}$  where  $x_0, \dots, x_{k-1}$  are the letters of the string, and  $a$  is a non-zero constant.

For the string badger:

- Write the polynomial to calculate the hash code for the string, where the constant  $a = 33$ , and the values for  $x_i$  correspond to the alphabetical positions of each letter ( $a = 1, b = 2$ , etc.).
- Rewrite your answer to (a) to factor the polynomial using **Horner's rule**.
- Calculate the hash code for badger.

## Question 8

Assume there is a hash table that contains integers represented by an array of capacity 13. The hash function for the table is  $x \bmod 13$ . The following values are to be inserted into the hash table in the given order: 70, 14, 31, 18, 20, 57

Starting with an initially empty table, sketch the hash table that results from inserting the values and handling collisions using the follow methods (as described in the textbook):

- a) **linear probing**
- b) **quadratic probing**
- c) **double hashing**. The second hash function is  $x \bmod 7$ .

## Questions 9

- a) Define  $\lambda$ , the **load factor** of a hash table.
- b) Can  $\lambda$  be greater than 1? Why or why not?
- c) What is **re-hashing**? When is re-hashing necessary?